Lecture 2
OS Basics

Prof. Yinqian Zhang

Spring 2025

What is a Computer?

- A computer is a digital electronic machine that can be programmed to carry out sequences of arithmetic or logical operations automatically.









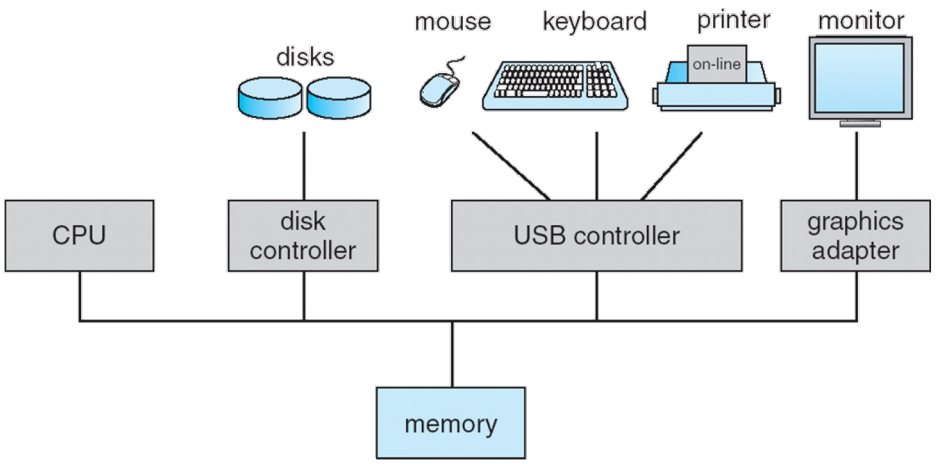


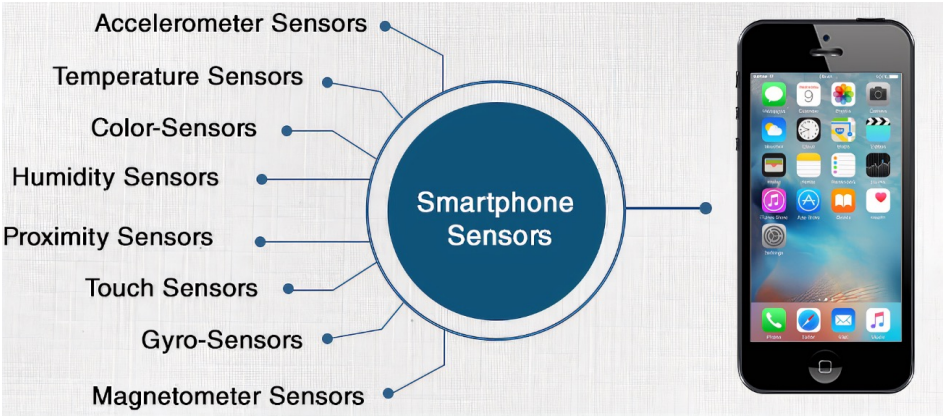




Computer System Organization

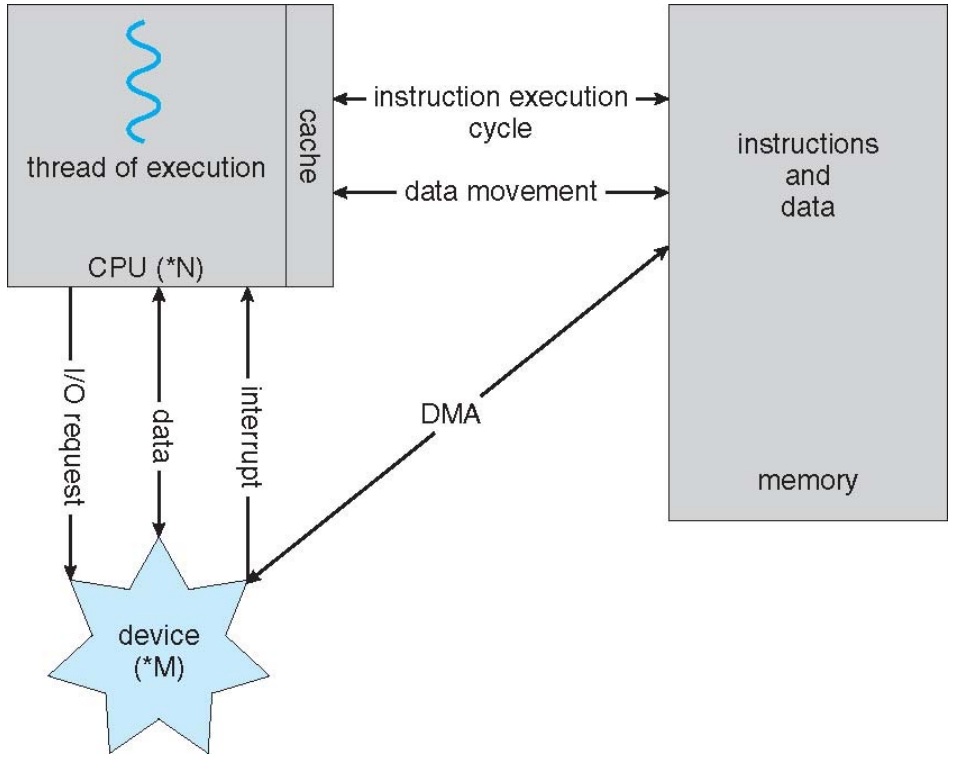
- One or more CPUs, device controllers connect through common bus providing access to shared memory
- Concurrent execution of CPUs and devices competing for memory cycles





How a Modern Computer Works

- von Neumann Architecture:
  - a single, shared memory for programs and data
  - a single bus for memory access
  - an arithmetic unit
  - and a program control unit



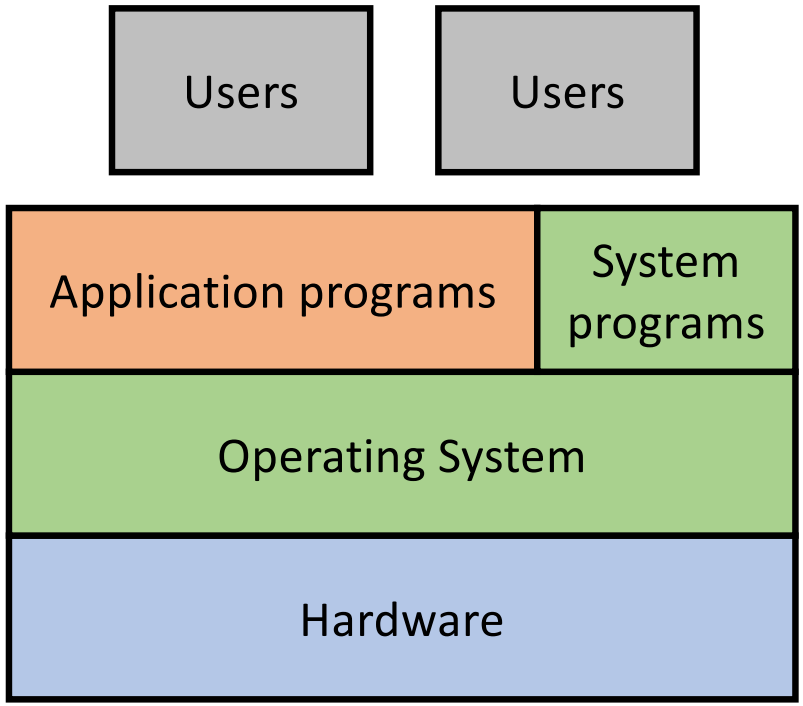
John von Neumann (1903~1957)
- Mathematician, computer scientist, physicist, chemist
- Known for invention of modern computer architecture and game theory

Structure of a Computer System

- Computer system can be divided into four components
  - Hardware
    - Provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs
    - Define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers

Structure of a Computer System

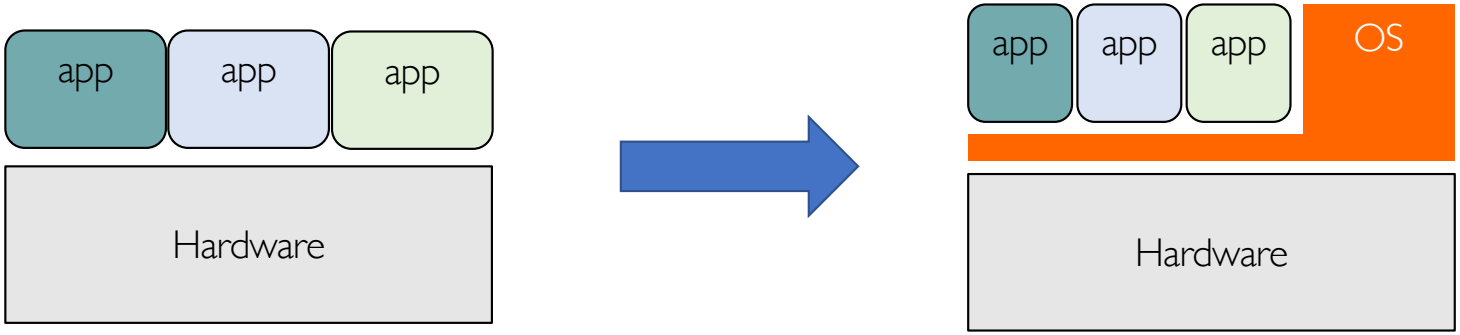
- Hardware
- Operating system
- Application programs
- Users



What is an Operating System?

Operating system was once called supervisor or master control program !!!

- A group of software that makes the computer operate **correctly** and **efficiently** in an easy-to-use manner.
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner (**hardware abstraction**)



What is an Operating System?

- Which one do you use: MacOS, Windows, or Linux?



Survey:
What OS do you use everyday on your laptop/desktop?

A: Windows
B: Mac OS
C: Linux

What's an Operating System?

- It includes a software program called **kernel**
  - manages all the physical devices (e.g., CPU, RAM and hard disk)
  - exposes some functions such as **system calls** for others to configure the kernel or build software (e.g., C library) on top

- It includes other "helper" programs
  - Such as a **shell**, which renders a simple command-line user interface with a full set of commands
  - Such as a **GUI** (graphic user interface), which renders a user-friendly interface with icons representing files and folders
  - Such as a **Browser**, which helps the user to visit websites

What's an Operating System?

- An OS is a **resource manager**
  - Managing CPUs, memory, disks, I/O devices (keyboards, USB drive, sensors, …)
  - Arbitrator of conflicting requests for efficient and fair resource use

- An OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer

What Does an Operating System Do?

- Virtualization
  - Virtualize CPU: Run multiple programs on a single CPU (as if there are many CPUs)
  - Virtualize memory: Give each process (or programs if you will) the illusion of running in its own memory address space
- Concurrency
  - Run multi-threaded programs and make sure they execute correctly
- Persistence
  - Write data (from volatile SRAM/DRAM) into persistent storage
  - Performance, crash-resilience
- ……

Evolution of OS

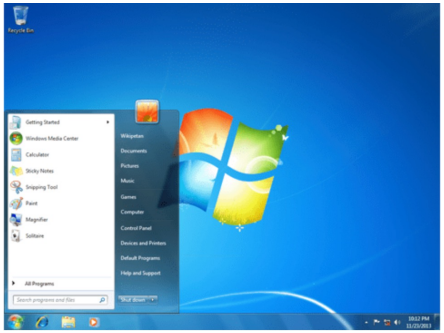
- Early OS: just a library to handle low-level I/O
- Atlas computing system: **system calls** that raise the hardware privilege level
  - Special instruction to transit between kernel mode and user mode
- UNIX: support of multi-programming and memory protection
- PC era: better security and useability
  - Disk Operating System (DOS), Mac OS, Windows, Linux
- Smart phones: user-facing applications, more sensors
  - iOS, Android, …

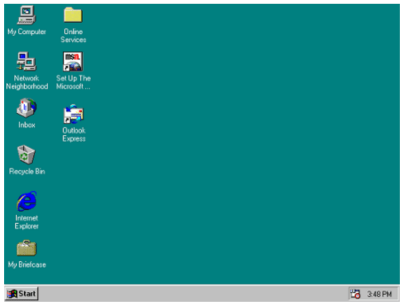
A Brief History of UNIX

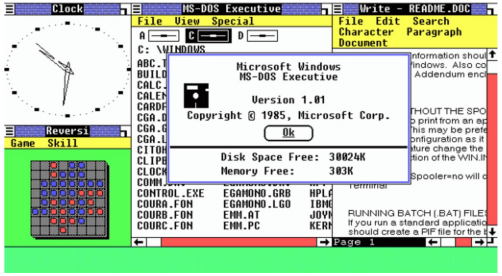
- Influenced by Multics system from MIT (https://multicians.org/)
- Originally by Ken Thompson and Dennis Ritchie at Bell Labs
  - Support meta-level programming with shell and pipe
  - Written in easy-to-understand C programming language
- Evolves to Berkeley Systems Distribution (BSD)
  - Advanced virtual memory, file system, and networking subsystems
- Commercial versions of UNIX
  - SunOS from Sun Microsystems, AIX from IBM, HPUX from HP, and IRIX from SGI.
- Mac OS has UNIX at its core
- Ideas and pricinples of UNIX inspire Linus Torvalds
  - The Linux Operating System!!

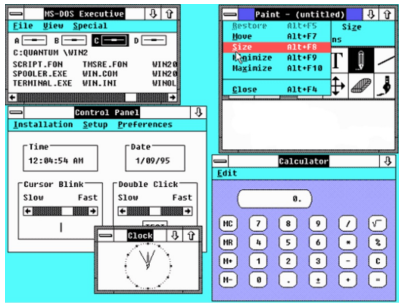
A Brief History of Windows

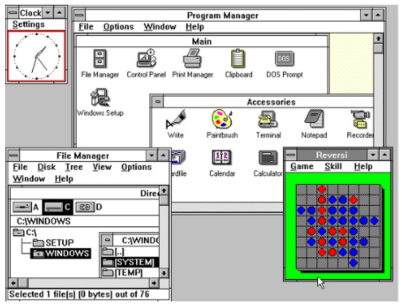
- Windows 1 (1985): Graphic user interface on MS-DOS
- Windows 2 (1987): Support overlaping windows
- Windows 3 (1990): Run MS-DOS programs on Windows
- Windows 3.1 (1992): TrueType fonts support
- Windows 95 (1995): Start menu and button
- Windows 98, ME, 2000, XP, Vista, 7, 8, 8.1, 10

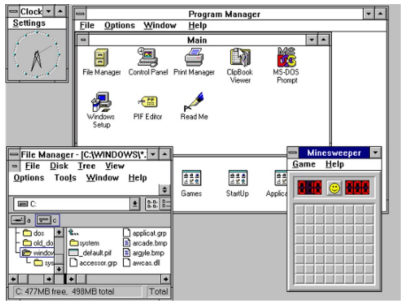








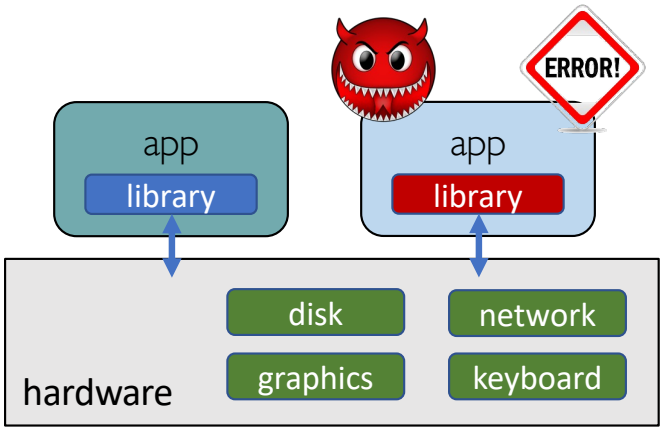


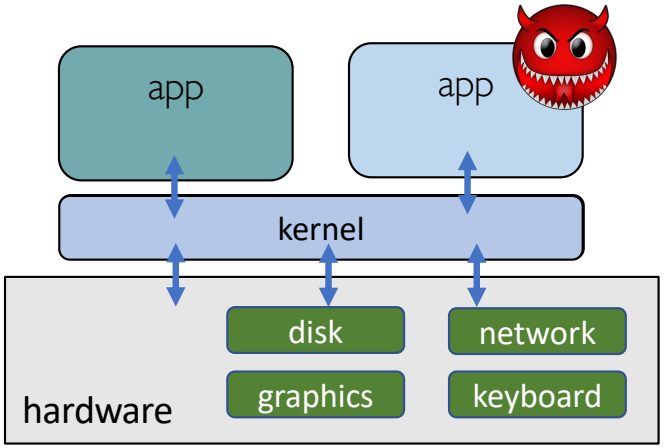


Dual-mode Operations

Evolution of Operating Systems

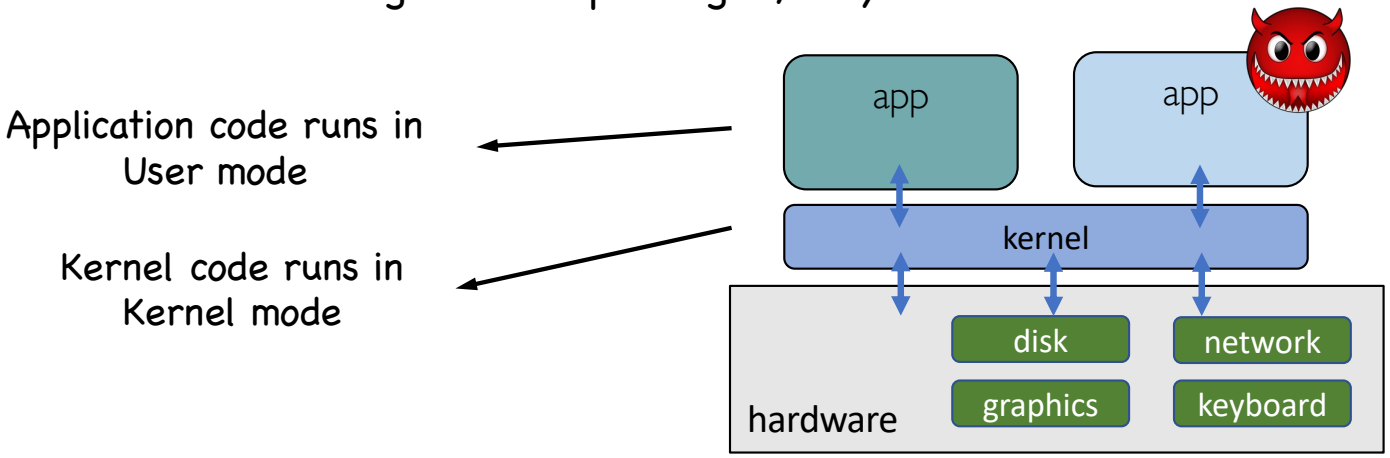
- A library to handle low-level I/O
  - Issue: Fault and security isolation
- Kernel: A bigger "library" to handle low-level I/O
  - Kernel needs to be protected from faulty/malicious apps





Kernel Mode vs. User Mode

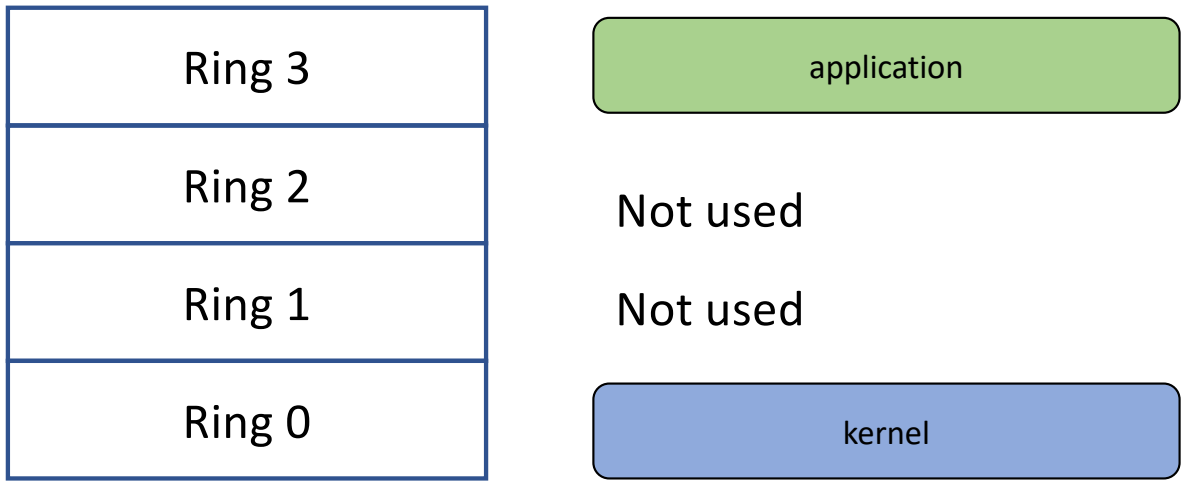
- Dual-mode operation allows OS to protect itself and other system components
  - Mode bits provided by CPU hardware
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions designated as privileged, only executable in kernel mode



Dual-mode Operation

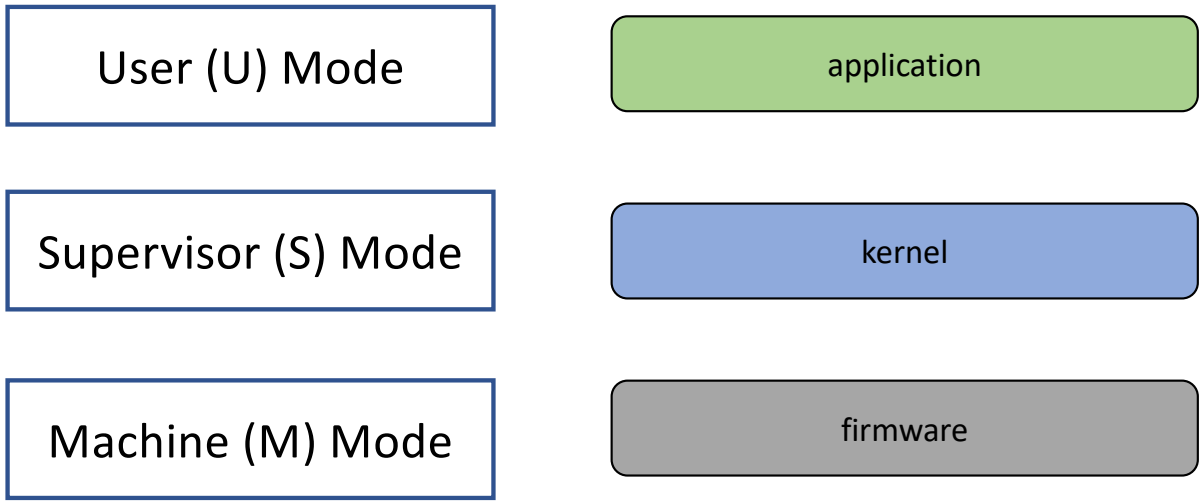
- Hardware provides at least two modes:
  - "Kernel" mode: Run kernel code
  - "User" mode: Normal programs executed
- What is needed in the hardware to support "dual mode" operation?
  - A bit for representing current mode (user/kernel mode bit)
  - Certain operations / actions only permitted in kernel mode
    - In user mode they fail or trap
  - User → Kernel transition *sets* kernel mode AND saves the user PC
    - Operating system code carefully puts aside user state then performs the necessary operations
  - Kernel → User transition *clears* kernel mode AND restores appropriate user PC

Mode Bits in CPUs



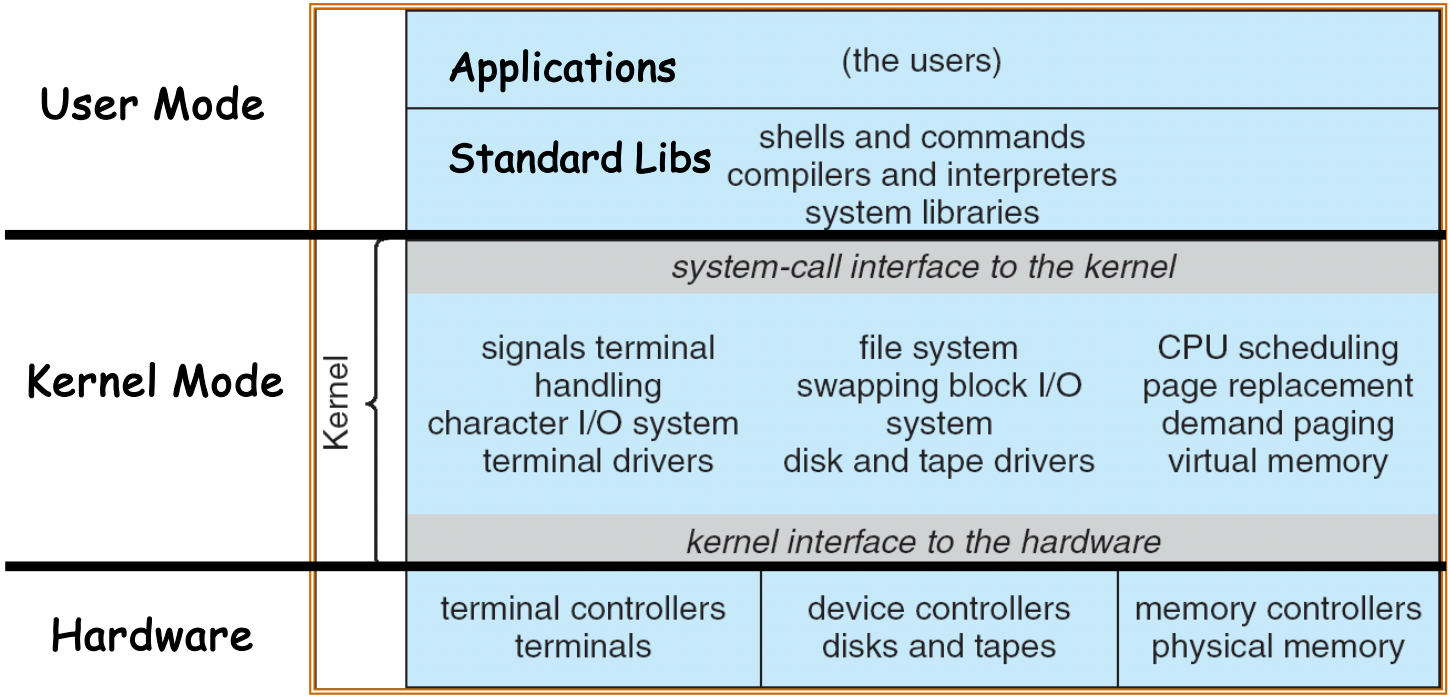
x86
(Intel & AMD)

Mode Bits in CPUs (Cont'd)



RISC-V

Unix System Structure

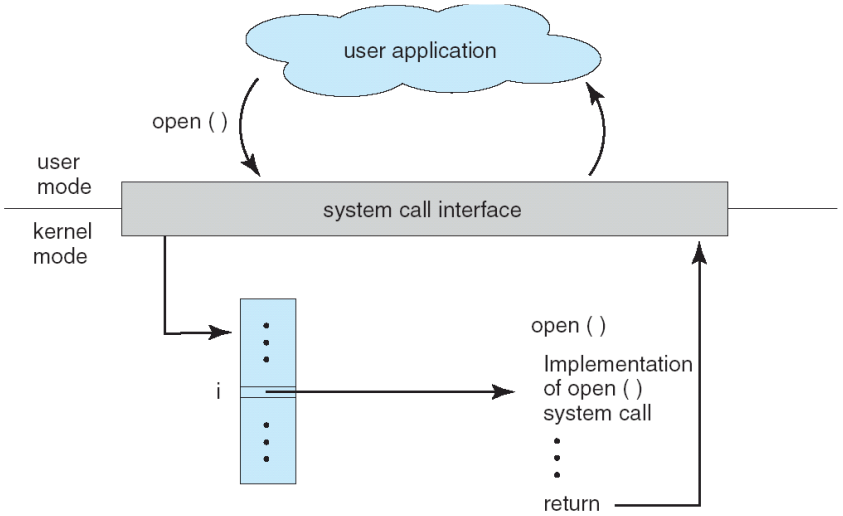


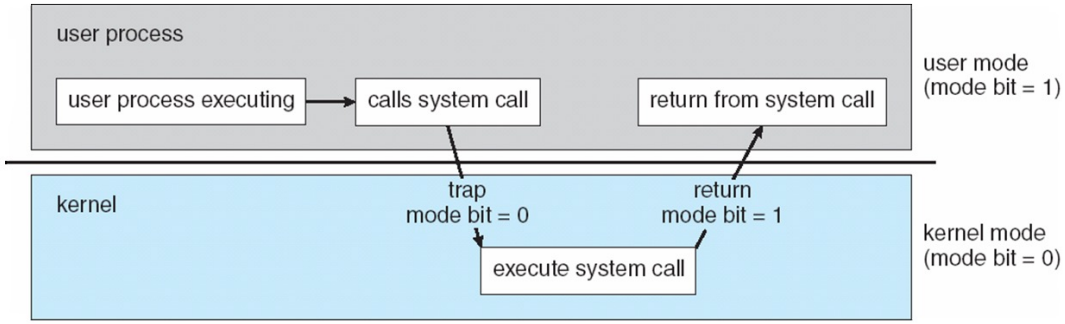
3 types of Mode Transitions

- System call
  - Process requests a system service, e.g., exit
  - Like a function call, but "outside" the process
  - Does not have the address of the system function to call
  - Marshall the syscall id and args in registers and exec syscall
- Interrupt
  - External asynchronous event triggers context switch
  - e. g., Timer, I/O device
  - Independent of user process
- Trap or Exception
  - Internal synchronous event in process triggers context switch
  - e.g., Protection violation (segmentation fault), Divide by zero, …

System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)



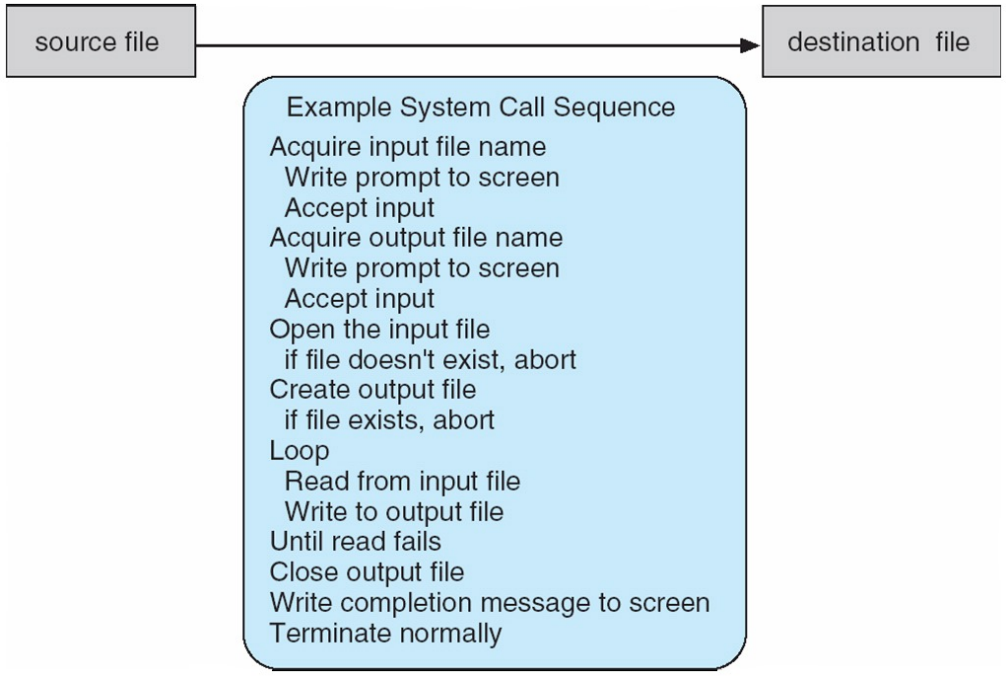


System Call Implementation

- Typically, a number associated with each system call
  - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller needs to know nothing about how the system call is implemented
  - Just needs to obey calling convention and understand what OS will do
  - Most details of OS interface hidden from programmer by library API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

Example of System Calls

- System call sequence to copy the contents of one file to another file



Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications
- Protection

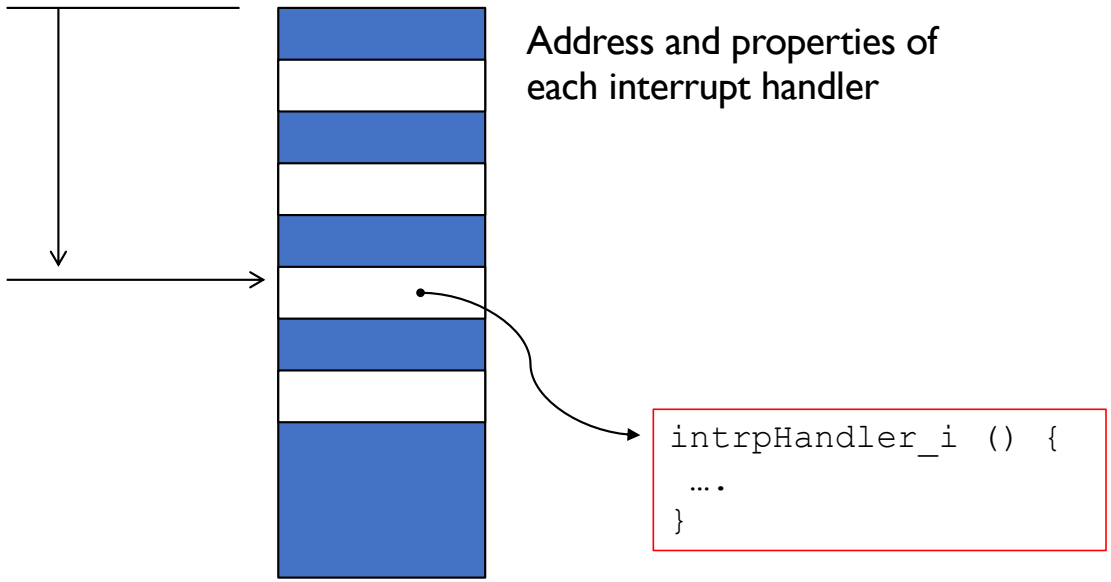
	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Exception and Interrupt

- Exceptions (synchronous) react to an abnormal condition
  - E.g., Map the swapped-out page back to memory
  - Divide by zero
  - Illegal memory accesses
- Interrupts (asynchronous) preempt normal execution
  - Notification from device (e.g., new packets, disk I/O completed)
  - Preemptive scheduling (e.g., timer ticks)
  - Notification from another CPU (i.e., Inter-processor Interrupts)

Exception and Interrupt (cont'd)

- Same procedure
  - Stop execution of the current program
  - Start execution of a handler
  - Processor accesses the handler through an entry in the Interrupt Descriptor Table (IDT)
  - Each interrupt is defined by a number



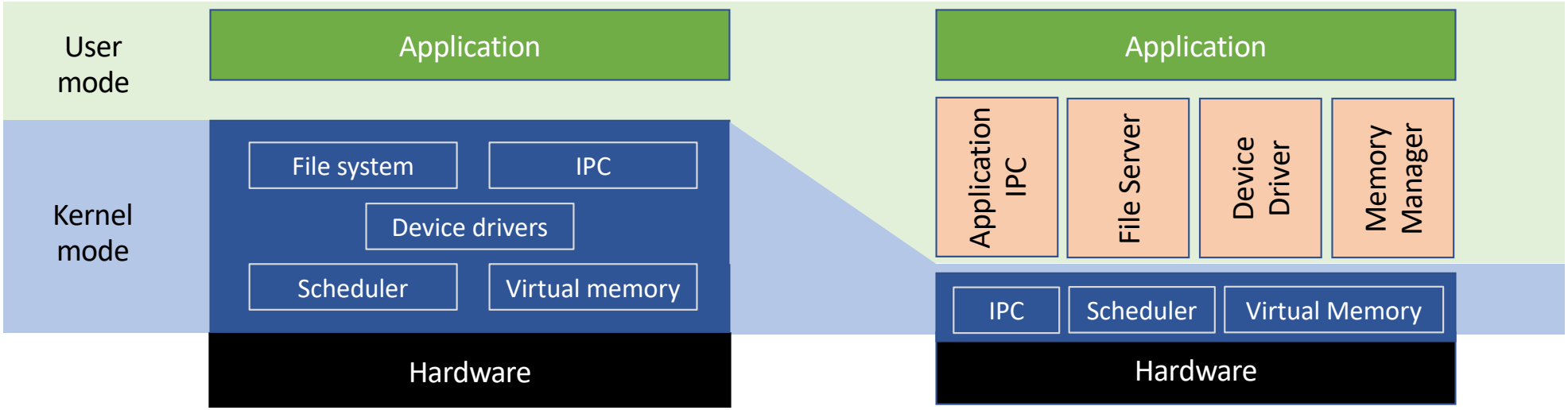
Kernel Structures

Monolithic Kernel

- A monolithic kernel is an operating system software framework that holds all privileges to access I/O devices, memory, hardware interrupts and the CPU stack.
- Monolithic kernels contain many components, such as memory subsystems and I/O subsystems, and are usually very large
  - Including filesystems, device drivers, etc.
- Monolithic kernel is the basis for Linux, Unix, MS-DOS.

Micro Kernel

- Microkernels outsource the traditional operating system functionality to ordinary user processes for better flexibility, security, and fault tolerance.



Micro Kernel (Cont'd)

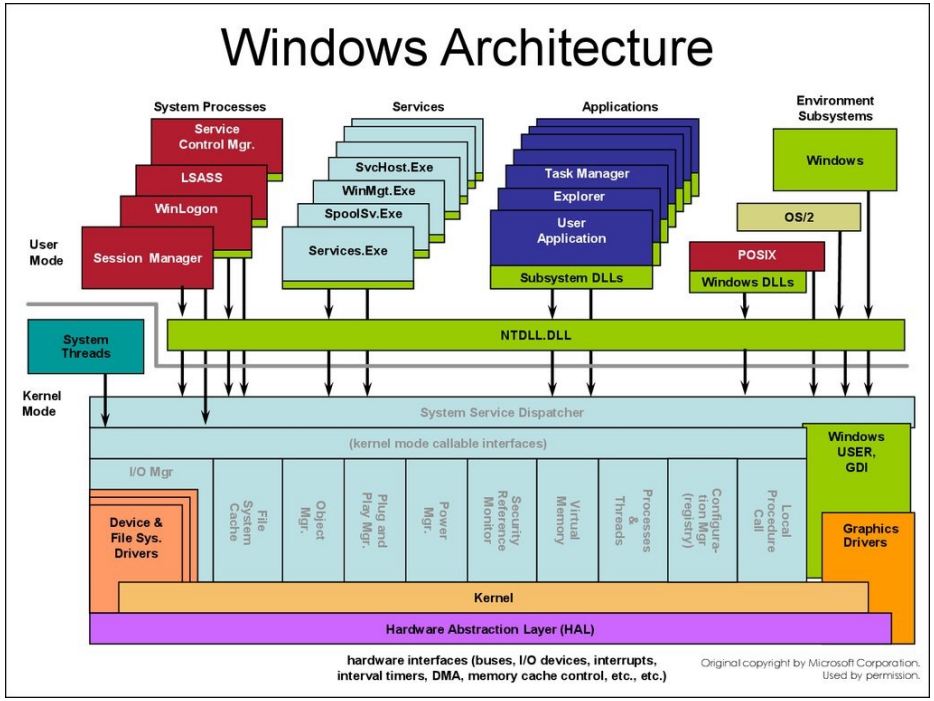
- OS functionalities are pushed to user-level servers (e.g., user-level memory manager)
- User-level servers are trusted by the kernel (often run as root)
- Protection mechanisms stay in kernel while resource management policies go to the user-level servers
- Representative micro-kernel OS
  - Mach, 1980s at CMU
  - seL4, the first formally verified micro-kernel, http://sel4.systems/

Micro Kernel (Cont'd)

- Pros
  - Kernel is more responsive (kernel functions in preemptible user-level processes)
  - Better stability and security (less code in kernel)
- Cons
  - More IPC needed and thus more context switches (slower)
  - Portability issues (application APIs may differ)

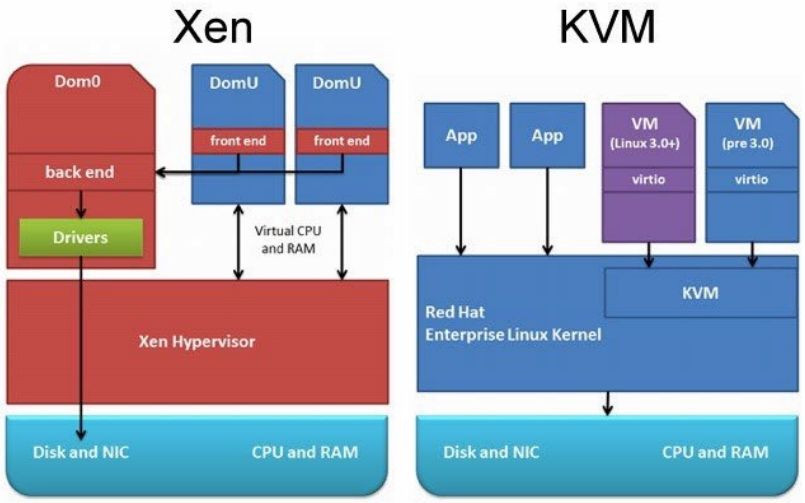
Hybrid Kernel

- A combination of a monolithic kernel and a micro kernel
  - Example: Windows OS



Virtualization and Hypervisors

- Hypervisor (or virtual machine manager/monitor, or VMM) emphasizes on **virtualization** and **isolation**
  - OS can run on hypervisor (almost) without modification
  - Resource partition among VMs
  - Micro kernel and exokernel can sometimes be used to implement hypervisors



OS Design Principles

- Internal structure of different Operating Systems can vary widely
  - Start by defining goals and specifications
  - Affected by choice of hardware, type of system
- User goals and System goals
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

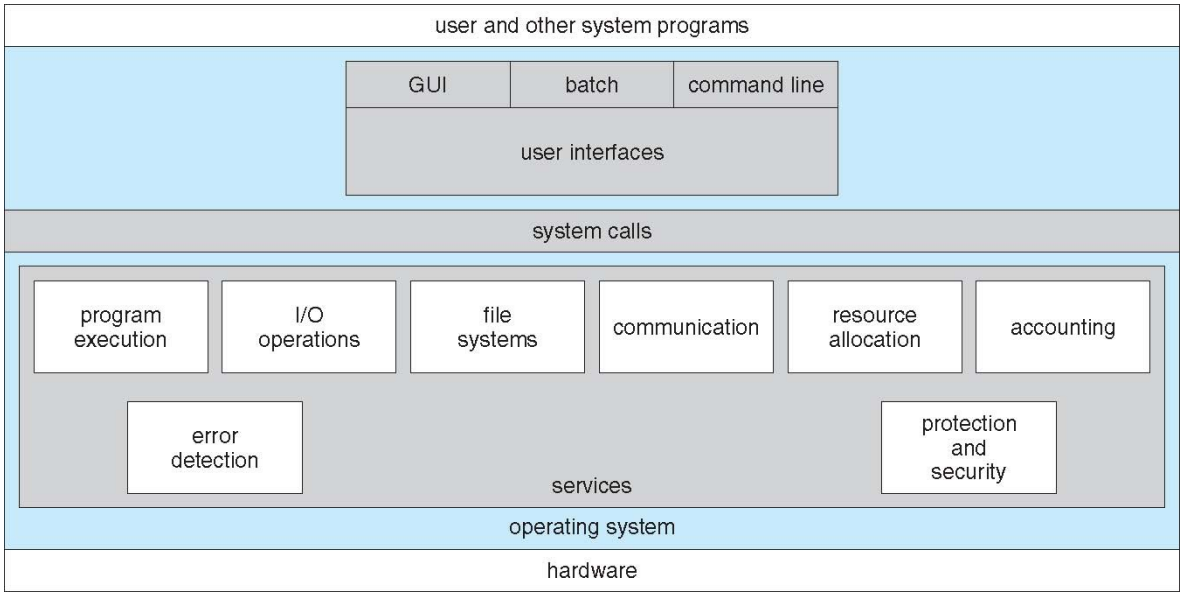
OS Design Principles

- OS separates policies and mechanisms
  - Policy: which software could access which resource at what time
    - E.g., if two processes access the same device at the same time, which one goes first
    - E.g., if a process hopes to read from keyboard
  - Mechanism: How is the policy enforced
    - E.g., request queues for devices, running queues for CPUs
    - E.g., access control list for files, devices, etc.
  - The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later

Operating System Services

Operating System Services

- Operating system provides a set of services to application programs



Operating System Services

- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (UI)
    - Varies between Command-Line (CLI), Graphics User Interface (GUI), Batch
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation** - The file system is of particular interest. Obviously, programs need to read and write files and directories, create and delete them, search them, list file Information, permission management

Operating System Services (Cont)

- One set of operating-system services provides functions that are helpful to the user (Cont):
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - **Error detection** – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Operating System Services (Cont)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
  - **Accounting** - To keep track of which users use how much and what kinds of computer resources
  - **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - Protection involves ensuring that all access to system resources is controlled
    - Security of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
    - If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link

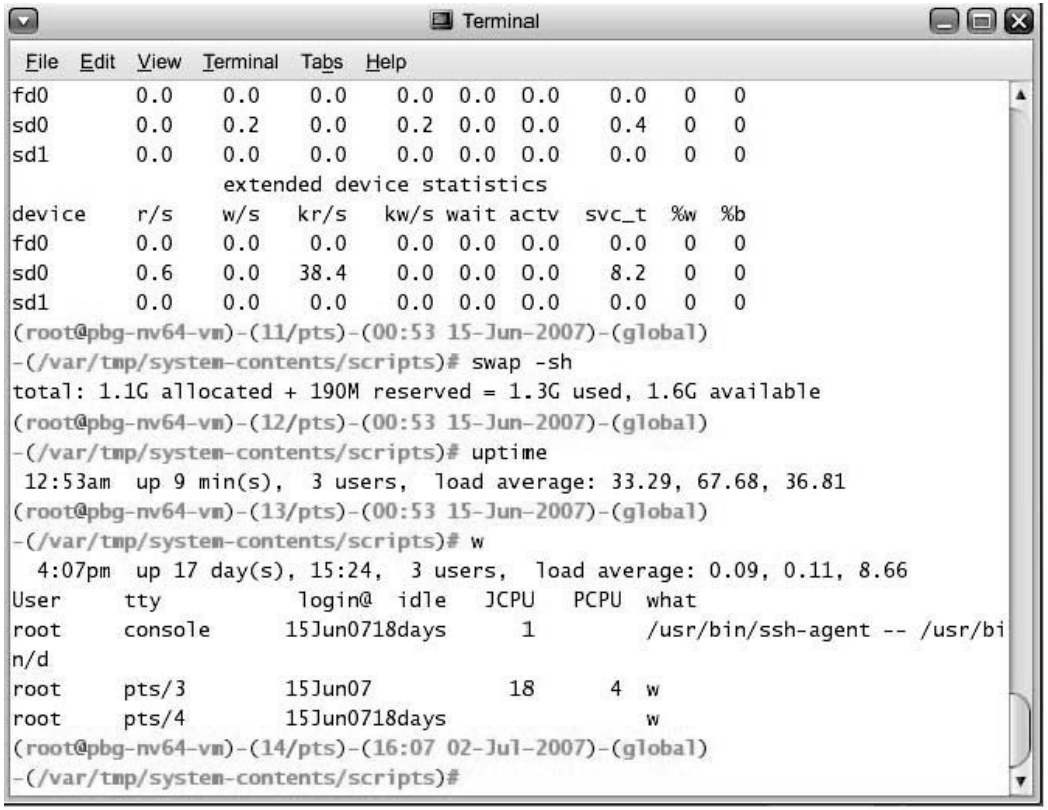
User Operating System Interface - CLI

- Command Line Interface (CLI) or command interpreter allows direct command entry
  - Sometimes implemented in kernel, sometimes by systems program
  - Shells: Bourne shell, C Shell, Bourne-Again Shell, Korn Shell
  - Primarily fetches a command from user and executes it
    - Sometimes commands built-in, sometimes just names of programs
    - If the latter, adding new features doesn't require shell modification

User Operating System Interface - GUI

- User-friendly desktop metaphor interface
  - Usually mouse, keyboard, and monitor
  - Icons represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions: provide information, options, execute function, open directory (known as a folder)
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI "command" shell
  - Apple Mac OS X as "Aqua" GUI interface with UNIX kernel underneath and shells available
  - Solaris is CLI with optional GUI interfaces (Java Desktop, KDE)

Bourne Shell Command Interpreter



The Mac OS X GUI



System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
  - File manipulation
  - Status information
  - File modification
  - Programming language support
  - Program loading and execution
  - Communications
  - Application programs
- Most users' view of the operation system is defined by system programs

System Programs (cont'd)

- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- File management - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- Status information
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a registry - used to store and retrieve configuration information

System Programs (cont'd)

- File modification
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- Programming-language support - Compilers, assemblers, debuggers and interpreters sometimes provided
- Program loading and execution
- Communications - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

Thank you!

